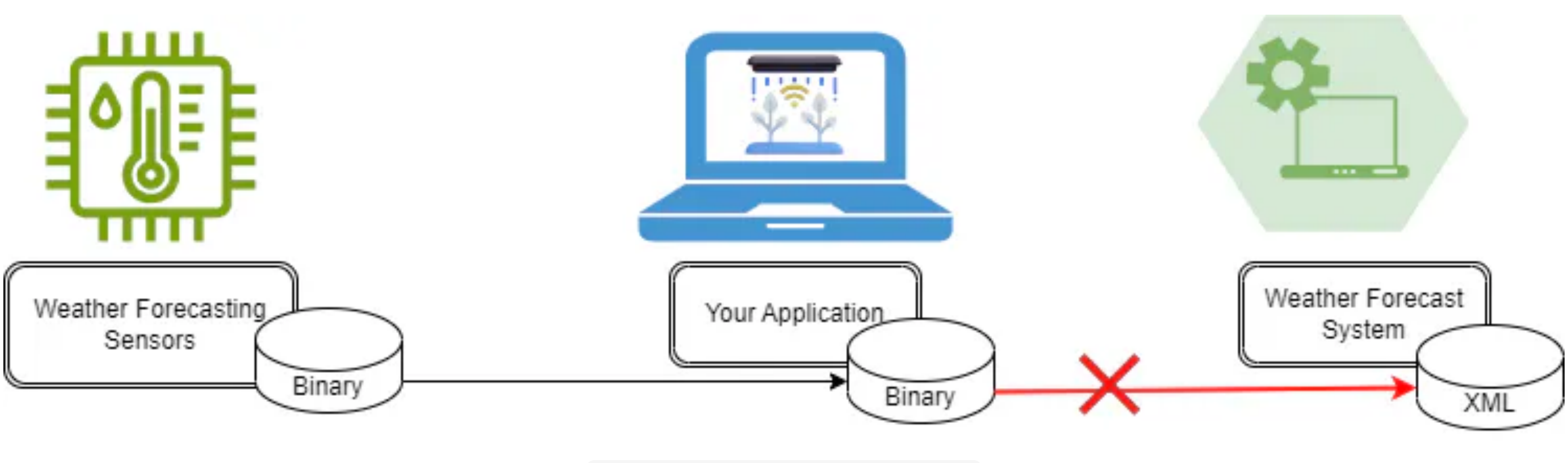
Adapter Pattern

Adapter Pattern is used when we have to make a relationship between two incompatible interfaces so that they can work together.

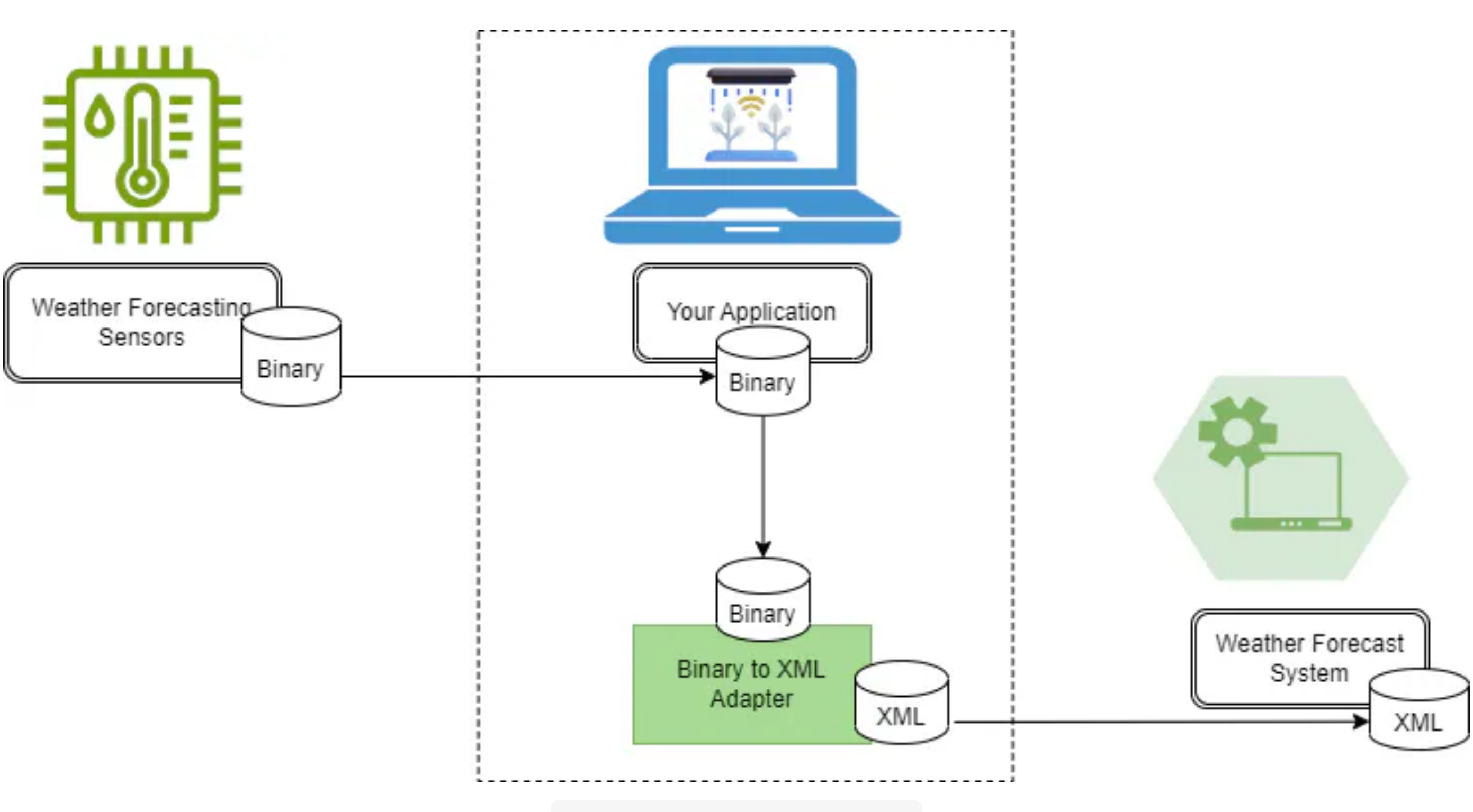
Assume that you are creating an application for weather forecasting. Your application collects weather information from several sensors in a proprietary binary format, which you display on a user-friendly interface. The following figure shows different components of this application.



Adapter Pattern - Problem

You choose to incorporate an advanced, third-party weather forecast system into your app to increase its accuracy. However, there is a twist: this method only accepts data in ordinary XML format, not the binary format provided by your sensors. The discrepancy in the data formats presents a problem.

Can you think of some solution to this problem? The solution is to create an Adapter. Here we go!

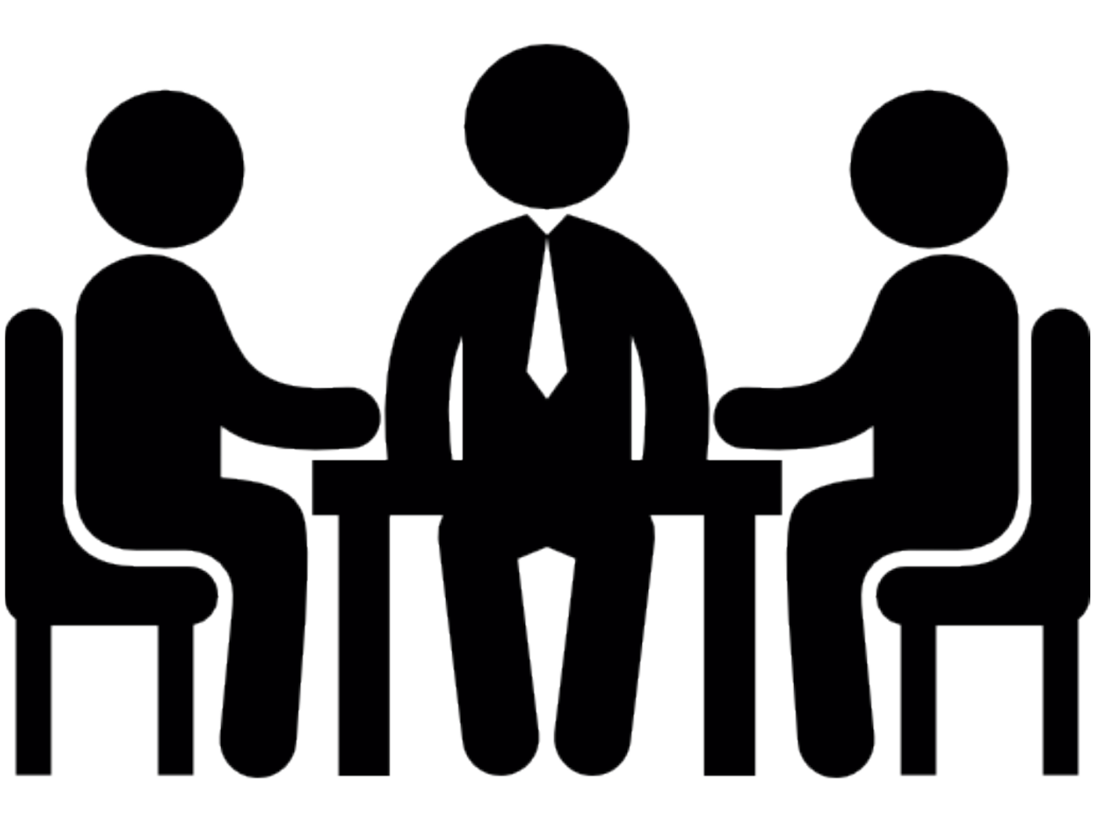


Adapter Pattern - Solution

This picture illustrates the connection between the weather forecast system, which needs XML format, and the weather data sensor, which generates data in a proprietary binary format. For the sensor and the prediction algorithm to work together seamlessly and be compatible, the Adapter is essential in transforming the binary input into XML. The above illustration makes it easier to see how the Adapter Pattern can be used to address the problem of incompatible data formats.

Real-world Example

To better understand the concept of the Adapter pattern, can you think of some real-world examples? Let's look at some example.



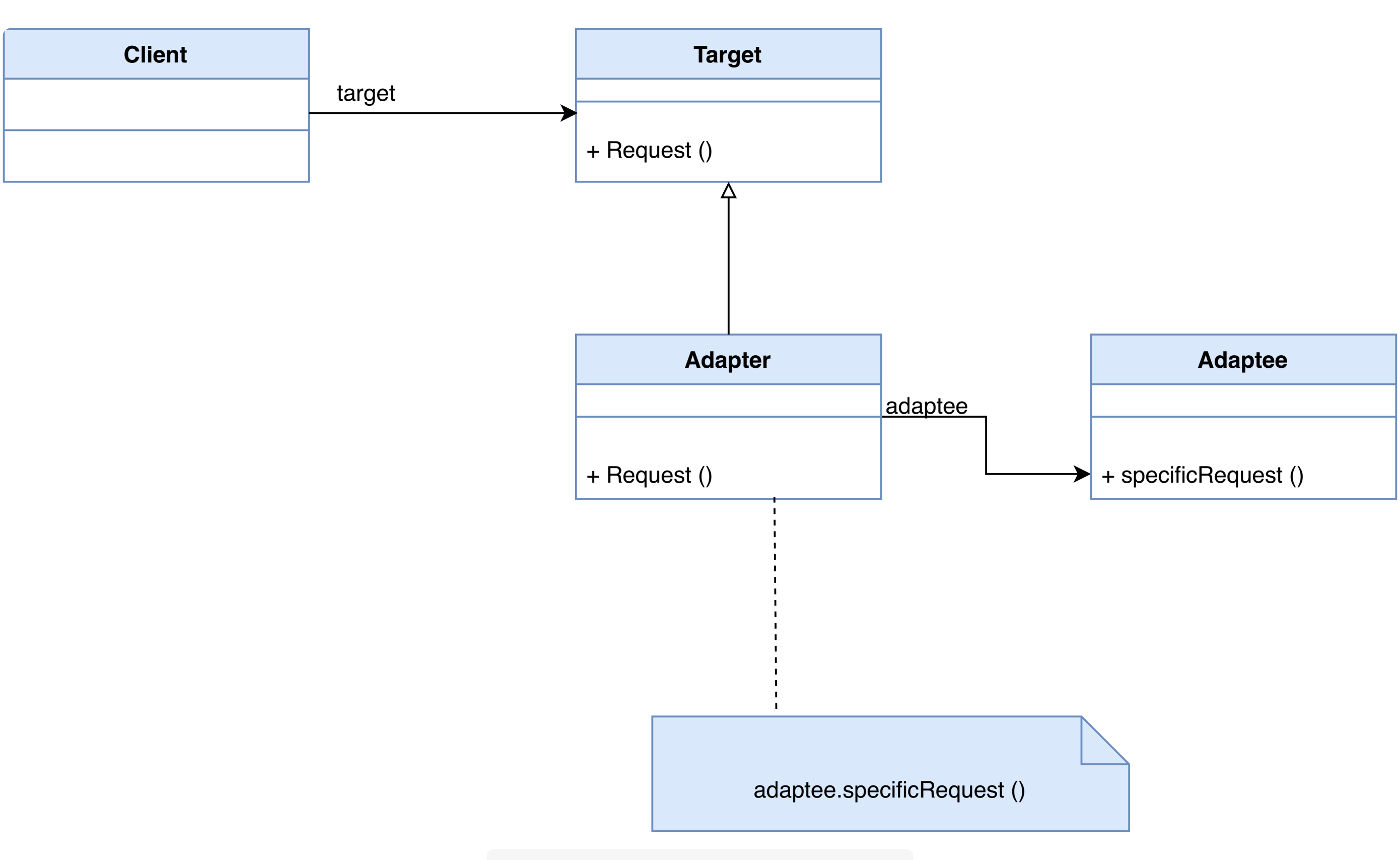
Adapter Pattern -Example

Here is a simple pictorial illustration showing a real-world example of the Adapter Pattern. The picture shows a view of a diplomatic meeting in which two diplomats, from different countries, speaking different languages, are communicating with each other with the help of a translator. Here, the translator translates the speech from both diplomats and acts as an adapter between two incompatible persons. This enables effective communication between the two parties despite their language barrier, just like the Adapter Pattern allows software components with incompatible interfaces to work together.

Structure of Adapter Pattern

The structure of the Adapter pattern can be shown using a class diagram. The components of class diagram are:

- **Client**: The class that contains the actual business logic. It interacts with the `Adapter` to use the service.
- **Target Interface**: This interface defines how the client wants to use the service. The `Adapter` class implements this interface.
- **Adapter**: This class implements the `Target` interface. It also contains a reference to the `Adaptee` class. It translates the interface from the `Target` to something the `Adaptee` understands.
- **Adaptee**: This is the class that implements the actual service that the `Client` wants to use but can't use directly because it has an incompatible interface with the `Client`.



Adapter Pattern - Class Diagram

Implementation of Adapter Pattern

Pseudocode

Let's implement the previously mentioned translator example and have a look at its pseudocode.

```
// Adaptee
CLASS FrenchSpeaker
    METHOD speakFrench(message)
        PRINT "Speaking in French: " + message
    END METHOD
END CLASS

// Target Interface
INTERFACE EnglishSpeaker
    METHOD speakEnglish(message)
END INTERFACE

// Adapter
CLASS Translator IMPLEMENTS EnglishSpeaker
    PRIVATE frenchSpeaker: FrenchSpeaker

    CONSTRUCTOR Translator(frenchSpeaker: FrenchSpeaker)
        this.frenchSpeaker = frenchSpeaker
    END CONSTRUCTOR

    METHOD speakEnglish(message)
        frenchMessage = translateToFrench(message)
        frenchSpeaker.speakFrench(frenchMessage)
    END METHOD

    PRIVATE METHOD translateToFrench(message)
        // Simplified translation logic
        RETURN message with "Hello" replaced by "Bonjour" and "Thank you" replaced by "Merci"
    END METHOD
END CLASS

// Client
CLASS EnglishClient
    PRIVATE speaker: EnglishSpeaker

    CONSTRUCTOR EnglishClient(speaker: EnglishSpeaker)
        this.speaker = speaker
    END CONSTRUCTOR

    METHOD express(message)
        speaker.speakEnglish(message)
    END METHOD
END CLASS

// Main
MAIN
    frenchSpeaker = NEW FrenchSpeaker()
    translator = NEW Translator(frenchSpeaker)
    client = NEW EnglishClient(translator)

    client.express("Hello! Thank you for the meeting.")
END MAIN
```

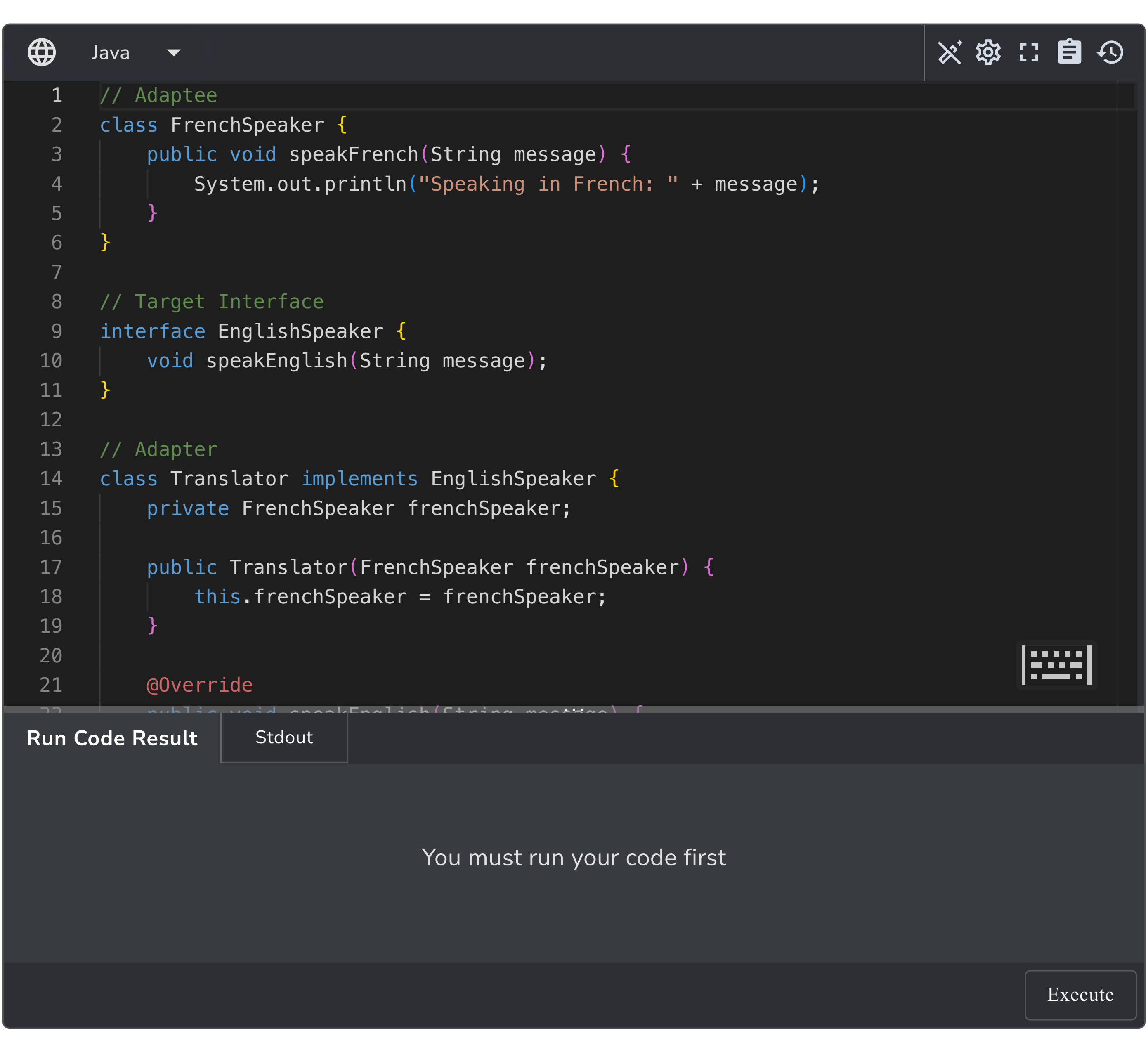
In this example, the Adapter Pattern is used to bridge the gap between the `EnglishClient` (which expects to communicate in English) and the `FrenchSpeaker` (which only understands French). The `Translator` acts as the adapter, translating English to French, allowing these two components to work together seamlessly. This demonstrates the power of the Adapter Pattern in integrating systems with incompatible interfaces.

- `FrenchSpeaker` is the Adaptee, providing functionality in French.
- `EnglishSpeaker` is the Target Interface.
- `Translator` is the Adapter, converting English messages to French.
- `EnglishClient` is the Client, using the services of `EnglishSpeaker`.

The `Translator` adapts the English messages to French, allowing the English-speaking client to effectively communicate with the French speaker.

Now, we will look at the implementation of this example in different programming languages.

Implementation



```java
// Adaptee
class FrenchSpeaker {
    public void speakFrench(String message) {
        System.out.println("Speaking in French: " + message);
    }
}

// Target Interface
interface EnglishSpeaker {
    void speakEnglish(String message);
}

// Adapter
class Translator implements EnglishSpeaker {
    private FrenchSpeaker frenchSpeaker;

    public Translator(FrenchSpeaker frenchSpeaker) {
        this.frenchSpeaker = frenchSpeaker;
    }

    @Override
```

You must run your code first

Application of Adapter Interface

We have understood the concept of the Adapter Pattern. But, when and where to use it? Let's discuss some of the applicability scenarios where adapter patterns can be used.

- **Incompatible Interfaces**:

  Use it when you have to communicate between two incompatible interfaces.

- **Legacy Code Integration**:

  The Adapter Pattern can also be used when you need to reuse some existing or legacy code that can't be altered. An adapter pattern can help to integrate it with new code.

- **Third-party Libraries**:

  If you are using any third-party library that is not compatible with your code, an adapter pattern can help you to bridge this gap.

- **Refactoring**:

  When you are refactoring some large codebase, adapters can help in working on new code with the old one very smoothly.

- **Multiple Data Formats**:

  If your application needs data from multiple sources and data is in multiple formats, adapters can help in standardising the data in a single format that can seamlessly work with your application.

Pros and Cons

Here's a table summarizing the pros and cons of the Adapter Pattern:

Pros	Cons
Increased Compatibility	**Increased Complexity**
it allows objects with different interfaces to collaborate with each other.	It adds extra classes that increase the complexity of the codebase.
Code Reusability	**Performance Overhead**
It is possible to reuse pre-existing code, even if the interfaces do not match with your system.	The extra layer of abstraction may affect performance, especially where speed is critical.
Single Responsibility Principle	**Not a Complete Solution for Incompatibility**
It keeps the interface interaction logic separate from the main business logic.	This is more of a temporary solution that doesn't tackle the root cause of the interface incompatibility issue.
Flexibility and Scalability	**Limited to Interface Adaptation**
It makes the code flexible and scalable as introducing new adapters to existing code is easy without changing it.	Instead of adapting the entire object's behaviour, it only adapts interfaces.
Simplifies the Client Interface	**Potential Overuse**
It hides the underlying complexity of interfaces, thus providing a simplified client interface.	It involves the risk of using adapters for even minor mismatches which increases complexity.

← Previous
Introduction

Next →
Bridge Pattern

☐ Mark as Completed